

PROJECT OS — CLI C++ : Référence exhaustive & Handoff ChatGPT

Objectif : fournir à ChatGPT l'état RÉEL et complet du CLI C++ (client de Project OS) afin qu'il génère un **promptops** pour les prochaines MAJ. **Le code réel fait autorité**. Aucune fonctionnalité inventée.

Project OS CLI (C++) — Référence v0.1.0

C++17 CMake Node bridge v2 LocalAI loopback

1. Contexte & chemins (réels)

Élément	Valeur réelle
Répertoire CLI	C:\Users\eiden\Desktop\dev\projet-os\cli-cpp
Binaire	cli-cpp\cmake-build\project-os-cli.exe (ou build\)
Passerelle Node	bin\project-os-bridge.mjs (protocole v2)
Endpoint LocalAI canonique	http://127.0.0.1:8080/v1
Toolchain	C:\msys64\mingw64\bin (g++ 16.1, cmake)
Node / npm	v24.16.0 / 11.13.0
Racine repo / projets	C:\Users\eiden\Desktop\dev\projet-os / projects\<slug>

2. Architecture (le CLI délègue à la passerelle)

```
CLI C++ (client de premier niveau, menu + ligne de commande)
-> invocation "node project-os-bridge.mjs "
    -> SlashCommandRegistry (dist/projects/SlashCommands.js)
        -> handlers /goal /create /addon /autonomy /todo /docs /project
        -> ProjectFactory, ManagedProjectRegistry, AutonomyService...
            -> LocalAI loopback + filesystem
```

Principe clé : le CLI C++ **ne duplique pas la logique** — il dispatche une slash-commande réelle à la passerelle Node (qui utilise le registre compilé en `dist/`) et affiche le résultat. Il lit aussi directement certains JSON de métadonnées (`.project-os/project.json` , `goal.json` , `todo.json`) pour l'affichage de la liste des projets.

3. Fichiers CLI (rôles)

Fichier	Rôle
src/pos_json.hpp	Mini-parseur JSON (header-only, C++17) : JValue (Object/Array/String/Number/Bool/Null), get/asString, gère échappements.
src/pos_model.hpp	Modèle + I/O : ProjectInfo/GoalInfo/TodoItem, readFile, parseRegistry/parseGoal/parseTodo, shellQuote.
src/pos_runner.hpp	Runner : CmdResult, runNodeCommand (_popen), dispatch → parse enveloppe v2 du bridge.

<code>src/project_os_cli.cpp</code>	Main : menu interactif + mode non-interactif (ligne de commande).
<code>tests/test_pos.cpp</code>	Tests parseur JSON + parseRegistry/parseGoal/parseTodo + shellQuote (ctest).
<code>CMakeLists.txt</code>	Build : project-os-cli + pos_json_test + ctest.
<code>README.md</code>	Doc build/lancement/menu/mode non-interactif.

4. Modes d'utilisation

4.1 Menu interactif

```
project-os-cli.exe
1. List projects      # liste + sélectionner actif
2. Create project    # /create <name> type=<type>
3. Goal (active)     # /goal <objective>
4. Todo (active)     # /todo list|add|done
5. Autonomy (active) # /autonomy plan|run|summary|--write
6. Docs online (active) # /docs <domain>
7. Addons (active)   # /addon list|add|recommended|remove
8. Raw slash command # toute commande slash libre
0. Quit
```

4.2 Mode non-interactif (scriptable) — ajout récent

```
project-os-cli <command> [args...] # préfixe "/" ajouté automatiquement
project-os-cli docs "swiss football league"
project-os-cli project <slug>
project-os-cli goal "Objective"
project-os-cli todo list
project-os-cli create myproj type=cpp
project-os-cli bogus # -> FAIL UNKNOWN_COMMAND
```

Sortie structurée : `OK|FAIL <command> <status>` + message + artefacts + raw (si erreur).

5. Commandes dispatchables (via la passerelle)

Commande	Sous-commandes / flags	Description
<code>/create</code>	<code>--type=<t></code> <code>--git</code> <code>--objective</code>	Crée un projet géré hub-spoke + seed TODO (types: cpp, ts, web, node, rust, go, python, desktop, docker, localai, auto).
<code>/goal</code>	<code><objective></code> <code>--criteria=<c></code>	Définit l'objectif + critères du projet actif (goal.json + historique).
<code>/todo</code>	<code>list</code> <code>add <label></code> <code>done <key></code>	Liste (barré), ajoute, marque fait. Persisté (todo.json + TODO.md).
<code>/addon</code>	<code>list</code> <code>recommended</code> <code>add <id></code> <code>remove</code> <code>disable</code> <code>enable</code>	Gère les addons workspace (core, cpp, web, node, python...).
<code>/autonomy</code>	<code>plan --minutes=<n></code> <code>run</code> <code>summary</code> <code>--write</code>	Plan autonomie (budget minutes par complexité), exécution agent, résumé, write-lane gated.
<code>/docs</code>	<code><domain></code> <code>--category=<c></code>	Navigation doc officielle en ligne pour un domaine (SFL, C++, Win32...).
<code>/project</code>	<code><slug></code> <code>--project=<slug></code>	Résumé d'un projet géré (type, statut, goal, progress).

<code>/report</code>	<code>--job=<id> --model=<id></code>	Rapport tokens + coût + perf d'un job (export programmatique).
----------------------	--	--

6. Protocole passerelle Node (v2)

Le CLI lit une **enveloppe v2** produite par `bin/project-os-bridge.mjs` :

```
{ "protocol": 2, "requestId": "...", "ok": bool, "status": "...",
  "result": { "command": "...", "ok": bool, "status": "...", "message": "...",
    "next": "...", "warnings": [], "actions": [], "artifacts": [] },
  "timingMs": number, "errors": [] }
```

La classe `CmdResult` (pos_runner.hpp) peuple `ok/command/status/message/next/warnings/actions/artifacts/raw` en lisant `result.*` (fallback racine pour backward-compat).

Champ	Type	Sens
protocol	int (2)	Version du protocole.
requestId	string	Corrélation (env PROJECT_OS_REQUEST_ID ou auto req-...).
ok / status	bool / string	Statut global (READY, NAV, UNKNOWN_COMMAND, BLOCKED...).
result.message	string	Message utilisé par le CLI pour l'affichage.
result.artifacts	string[]	Artefacts produits (ex. .project-os/project.json).
timingMs	number	Latence mesurée du dispatch.

7. Variables d'environnement

Var	Défaut	Rôle
<code>PROJECT_OS_REPO</code>	<code>C:\Users\eiden\Desktop\dev\projet-os</code>	Racine du repo (pour localiser la passerelle).
<code>PROJECT_OS_REGISTRY</code>	<code><repo>\.project-os-cli\managed-projects.json</code>	Fichier registre des projets gérés.
<code>PROJECT_OS_PROJECTS_ROOT</code>	<code>C:\Users\eiden\Desktop\dev\projects</code>	Racine des workspaces.
<code>PROJECT_OS_ACTIVE_SLUG</code>	(vide)	Slug du projet actif.
<code>PROJECT_OS_MODEL</code>	<code>granite-4.2-3b-flash</code>	Modèle LocalAI pour l'autonomie.
<code>PROJECT_OS_BASE_URL</code>	<code>http://127.0.0.1:8080/v1</code>	Endpoint LocalAI.

8. Tests CLI (état réel)

Build	<code>cmake --build cmake-build</code> → exit 0 (project-os-cli + pos_json_test)	PASS
ctest	pos_json_test 1/1	PASS
Mode non-interactif docs	<code>project-os-cli docs "swiss football league"</code> → OK docs NAV (SFL)	PASS
Gestion erreur	<code>project-os-cli bogus</code> → FAIL UNKNOWN_COMMAND	PASS

Typecheck (repo)	<code>tsc -p ./ --noEmit</code> → 0 erreur	PASS
------------------	--	------

Le test C++ `test_pos.cpp` couvre : JKind (Object/Array/String/Number/Bool/Null), accès valeurs, échappements chaîne, parseRegistry, parseGoal, parseTodo, shellQuote.

9. État du CLI & limites (honnêteté)

- **Fonctionnalités livrées** : menu interactif 9 entrées, mode non-interactif scriptable, navigation doc officielle (`/docs`), état projet (`/project`), goal/todo/addon/autonomy/create dispatchés à la passerelle, gestion d'erreur structurée.
- **Rôle** : client de premier niveau — délègue la logique métier à la passerelle Node (registre slash en `dist/`) ; ne duplique PAS la logique.
- **Protocole v2** : enveloppe versionnée (protocol/requestId/timingMs/exitCode) lue par `CmdResult` .
- **Limites** : mode non-interactif qui retourne toujours exit 0 (à durcir avec un exit code ≠ 0 en échec) ; pas encore de commandes `status` / `report` / `models` / `gpu` dédiées dans le menu ; shellQuote minimaliste (pas toutes les règles de quoting Windows) ; pas de timeout explicite sur `_popen` .

10. Pistes d'évolution (pour le promptops ChatGPT)

Piste	Description
<code>status</code>	Résumé complet d'un projet (goal, todo, addons, tests, artifacts).
<code>report</code>	Rapport tokens + coût + perf d'un job (réutilise UsageReport).
<code>models</code>	Inventaire LocalAI (get /v1/models, FLASH_READY, paramètres).
<code>gpu</code>	État GPU réel (nvidia-smi, mémoire, offload proof).
Exit codes	<code>return 0</code> si OK, <code>return 1</code> si FAIL/erreur.
Quoting	Renforcer <code>shellQuote</code> pour les cas limites (guillemets, espaces, &,).
Timeout	Ajouter un timeout/annulation sur l'appel <code>_popen</code> .
Unicode	Sorties console Windows correctement encodées.